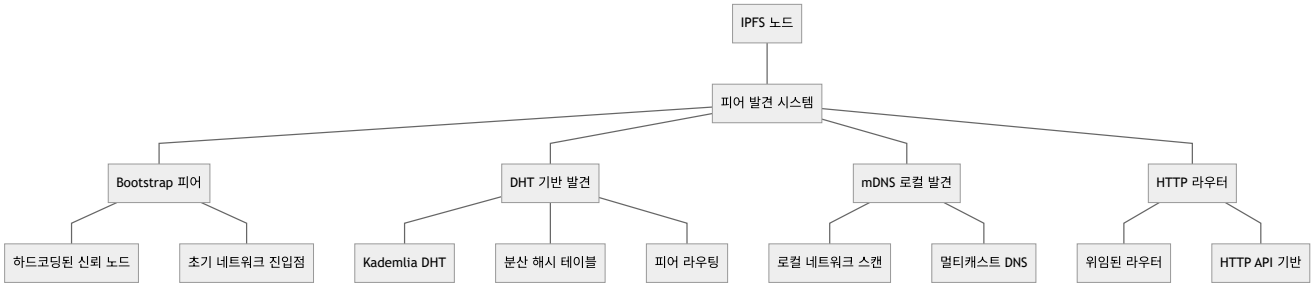
피어 발견 메커니즘 상세 분석

개요

IPFS 네트워크에서 피어 발견은 노드가 다른 노드들을 찾고 연결하는 핵심 메커니즘입니다. IPFS는 여러 가지 발견 방식을 조합하여 효율적이고 견고한 네트워크 연결성을 제공합니다. 이 문서는 DHT, mDNS, Bootstrap 피어를 통한 발견 과정과 각각의 특징을 상세히 분석합니다.

피어 발견 방식 개요



Bootstrap 피어 시스템

Bootstrap 피어의 역할

Bootstrap 피어는 IPFS 네트워크의 초기 진입점 역할을 합니다. 새로운 노드가 네트워크에 참여할 때 가장 먼저 연결을 시도하는 신뢰할 수 있는 노드들입니다.

기본 Bootstrap 피어 목록

```
var DefaultBootstrapAddresses = []string{
"/dnsaddr/bootstrap.libp2p.io/p2p/QmNnooDu7bfjPFoTZYxMNLWUQJyrVwtbZg5gBMjTezGAJN
"/dnsaddr/bootstrap.libp2p.io/p2p/QmQCU2EcMqAqQPR2i9bChDtGNJchTbq5TbXJJ16u19uLTa
"/dnsaddr/bootstrap.libp2p.io/p2p/QmbLHAnMoJPWSCR5Zhtx6BHJX9KiKNN6tpvbUcqanj75Nb
"/dnsaddr/bootstrap.libp2p.io/p2p/QmcZf59bWwK5XFi76CZX8cbJ4BhTzzA3gU1ZjYZcYW3dwt
"/dnsaddr/va1.bootstrap.libp2p.io/p2p/12D3KooWKnDdG3iXw9eTFijk3EWSunZcFi54Zka4wm
"/ip4/104.131.131.82/tcp/4001/p2p/QmaCpDMGvV2BGHeYERUEnRQAwe3N8SzbUtfsmvsqQLuvuJ
"/ip4/104.131.131.82/udp/4001/quic-v1/p2p/QmaCpDMGvV2BGHeYERUEnRQAwe3N8SzbUtfsmv
}
```

Bootstrap 과정

1. **노드 시작**: IPFS 노드가 시작되면 설정된 bootstrap 피어 목록을 로드
2. **연결 시도**: 각 bootstrap 피어에 순차적으로 연결 시도
3. **DHT 참여**: 성공적으로 연결된 bootstrap 피어를 통해 DHT 네트워크에 참여
4. **피어 정보 수집**: Bootstrap 피어로부터 다른 피어들의 정보 수집

Bootstrap 피어 파싱

```
func ParseBootstrapPeers(addrs []string) ([]peer.AddrInfo, error) {
maddrs := make([]ma.Multiaddr, len(addrs))
for i, addr := range addrs {
```

```
var err error
maddrs[i], err = ma.NewMultiaddr(addr)
if err != nil {
return nil, err
}
}
return peer.AddrInfosFromP2pAddrs(maddrs...)
}
```

Bootstrap 피어의 장단점

장점:

- 네트워크 진입의 신뢰할 수 있는 시작점
- 높은 가용성과 안정성
- 글로벌 분산으로 지역적 접근성 보장

단점:

- 중앙화된 의존성 (단일 실패점 가능성)
- 네트워크 검열 가능성
- 초기 연결 지연 가능성

DHT 기반 피어 발견

Kademlia DHT 구조

IPFS는 Kademlia 알고리즘을 기반으로 한 분산 해시 테이블(DHT)을 사용합니다:

```
graph TB
A[로컬 노드] --> B[DHT 라우팅 테이블]
B --> C[K-버킷 0]
B --> D[K-버킷 1]
B --> E[K-버킷 ...]
B --> F[K-버킷 159]
C --> C1[거리 2^0 ~ 2^1]
D --> D1[거리 2^1 ~ 2^2]
E --> E1[거리 2^i ~ 2^(i+1)]
F --> F1[거리 2^159 ~ 2^160]
```

DHT 모드

IPFS DHT는 세 가지 모드로 동작할 수 있습니다:

1. Auto Mode (기본값)

```
DHTOption RoutingOption = constructDHTRouting(dht.ModeAuto)
```

- 네트워크 환경에 따라 자동으로 클라이언트/서버 모드 선택
- NAT 뒤에 있으면 클라이언트 모드, 공개 IP가 있으면 서버 모드

2. Client Mode

```
DHTClientOption = constructDHTRouting(dht.ModeClient)
```

- DHT 쿼리만 수행, 다른 노드의 쿼리에 응답하지 않음
- 리소스 제약이 있는 환경에 적합

3. Server Mode

```
DHTServerOption = constructDHTRouting(dht.ModeServer)
```

- DHT 쿼리 수행 및 다른 노드의 쿼리에 응답
- 네트워크 인프라 역할 수행

Dual DHT 구조

IPFS는 WAN(Wide Area Network)과 LAN(Local Area Network)을 위한 이중 DHT 구조를 사용합니다:

```
func constructDHTRouting(mode dht.ModeOpt) RoutingOption {
return func(args RoutingOptionArgs) (routing.Routing, error) {
wanOptions := []dht.Option{
dht.BootstrapPeers(args.BootstrapPeers...),
}
lanOptions := []dht.Option{}
if args.LoopbackAddressesOnLanDHT {
lanOptions = append(lanOptions, dht.AddressFilter(nil))
}
return dual.New(
args.Ctx, args.Host,
dual.DHTOption(dhtOpts...),
```

```
dual.WanDHTOption(wanOptions...),
dual.LanDHTOption(lanOptions...),
)
}
}
```

WAN DHT

- 인터넷 전체의 IPFS 노드들과 연결
- Bootstrap 피어를 통한 초기 연결
- 글로벌 콘텐츠 발견 및 라우팅

LAN DHT

- 로컬 네트워크 내의 IPFS 노드들과 연결
- 로컬 주소 필터링 없이 동작 가능
- 빠른 로컬 콘텐츠 접근

DHT 피어 발견 과정

1. **라우팅 테이블 구축**: 가까운 피어들부터 시작하여 점진적으로 확장
2. **FIND_NODE 쿼리**: 특정 노드 ID에 가까운 피어들 검색
3. **FIND_VALUE 쿼리**: 특정 키에 대한 값 또는 제공자 검색
4. **주기적 갱신**: 라우팅 테이블의 피어 정보 주기적 갱신

DHT 최적화 기능

Accelerated DHT Client

```
if  dualDHT != nil && cfg.Routing.AcceleratedDHTClient.WithDefault(config.Defaul
fullRTClient, err := fullrt.NewFullRT(in.Host,
dht.DefaultPrefix,
fullrt.DHTOption(
dht.Validator(in.Validator),
dht.Datastore(in.Repo.Datastore()),
dht.BootstrapPeers(bspeers...),
dht.BucketSize(20),
),
)
}
```

- 전체 라우팅 테이블을 유지하여 더 빠른 검색 제공
- 메모리 사용량 증가 대신 검색 성능 향상
- 서버급 노드에 적합

mDNS 로컬 발견

mDNS 동작 원리

mDNS(Multicast DNS)는 로컬 네트워크에서 서비스를 자동으로 발견하는 프로토콜입니다:

```
func  SetupDiscovery(useMdns  bool) func(helpers.MetricsCtx, fx.Lifecycle, host.
return  func(mctx helpers.MetricsCtx, lc fx.Lifecycle, host host.Host, handler *
```

```
if  useMdns {
service := mdns.NewMdnsService(host, mdns.ServiceName, handler)
if  err := service.Start();  err != nil {
log.Error("error starting mdns service: ", err)
return  nil
}
}
return  nil
}
}
```

mDNS 발견 과정

1. **서비스 광고**: 노드가 로컬 네트워크에 자신의 존재를 멀티캐스트로 광고
2. **서비스 발견**: 다른 노드들이 멀티캐스트 메시지를 수신하여 피어 발견
3. **자동 연결**: 발견된 피어에 자동으로 연결 시도

```
func (dh *discoveryHandler) HandlePeerFound(p peer.AddrInfo) {
log.Info("connecting to discovered peer: ", p)
ctx, cancel := context.WithTimeout(dh.ctx, discoveryConnTimeout)
defer  cancel()
if  err := dh.host.Connect(ctx, p);  err != nil {
log.Warnf("failed to connect to peer %s found by discovery: %s", p.ID, err)
}
```

```
}
```

mDNS의 장단점

장점:

- 로컬 네트워크에서 빠른 피어 발견
- 인터넷 연결 없이도 동작
- 자동 설정으로 사용자 개입 불필요

단점:

- 로컬 네트워크로 제한됨
- 네트워크 트래픽 증가 가능성
- 일부 네트워크 환경에서 차단될 수 있음

HTTP 라우터 기반 발견

위임된 라우팅

IPFS는 HTTP API를 통한 위임된 라우팅을 지원합니다:

```
func  constructDefaultHTTPRouters(cfg *config.Config) ([]*routinghelpers.Paralle
var  routers []*routinghelpers.ParallelRouter
var  httpRouterEndpoints []string
if  os.Getenv(config.EnvHTTPRouters) != "" || len(cfg.Routing.DelegatedRouters)
httpRouterEndpoints = config.DefaultHTTPRouters
} else {
```

```
    httpRouterEndpoints = cfg.Routing.DelegatedRouters

    }

    for _, endpoint := range  httpRouterEndpoints {

    httpRouter, err := irouting.ConstructHTTPRouter(endpoint, cfg.Identity.PeerID,

    httpAddrsFromConfig(cfg.Addresses), cfg.Identity.PrivKey, httpRetrievalEnabled)

    if  err != nil {

    return  nil, err

    }

    // HTTP 라우터 설정...

    }

    }
```

HTTP 라우팅 API

HTTP 라우터는 다음 엔드포인트를 제공합니다:

- `GET /routing/v1/ipns/{name}` : IPNS 레코드 조회
- `PUT /routing/v1/ipns/{name}` : IPNS 레코드 게시
- `GET /routing/v1/peers/{peer-id}` : 피어 정보 조회
- `GET /routing/v1/providers/{cid}` : 콘텐츠 제공자 조회

HTTP 라우터의 장단점

장점:

- 빠른 응답 속도 (중앙화된 인덱스)
- 높은 가용성

- 웹 기반 노드와의 호환성

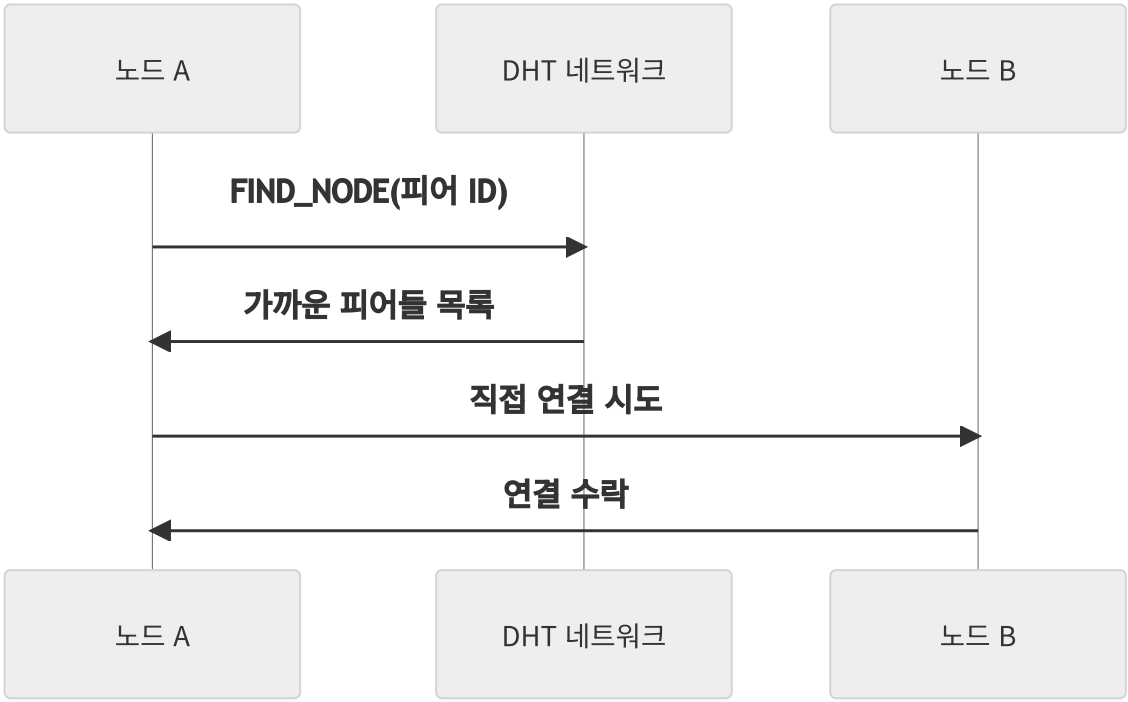
단점:

- 중앙화된 의존성
- 프라이버시 우려
- 검열 가능성

피어 라우팅 vs 콘텐츠 라우팅

피어 라우팅 (Peer Routing)

피어 라우팅은 특정 피어의 위치를 찾는 과정입니다:

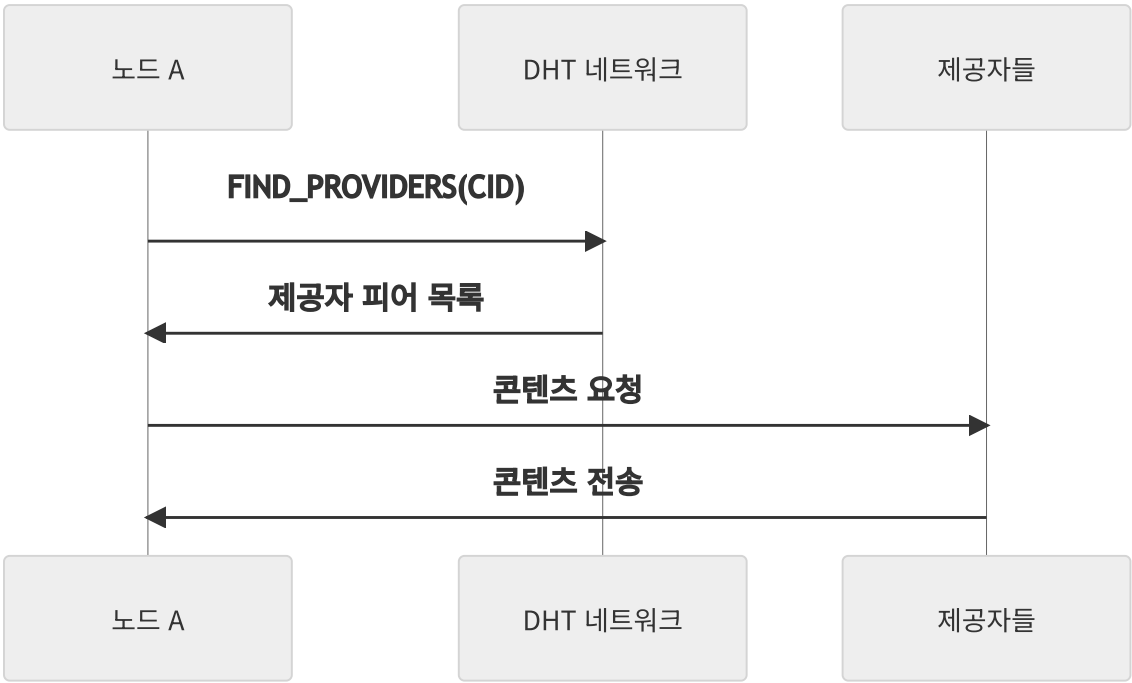


특징:

- 특정 피어 ID를 기반으로 검색
- 피어의 네트워크 주소 정보 반환
- 직접적인 P2P 연결 설정에 사용

콘텐츠 라우팅 (Content Routing)

콘텐츠 라우팅은 특정 콘텐츠를 제공하는 피어들을 찾는 과정입니다:



특징:

- 콘텐츠 ID(CID)를 기반으로 검색
- 해당 콘텐츠를 가진 피어들의 목록 반환
- Bitswap 프로토콜과 연계하여 콘텐츠 교환

통합 라우팅 시스템

병렬 라우터 구조

IPFS는 여러 라우팅 방식을 병렬로 실행하여 최적의 성능을 제공합니다:

```
func Routing(in p2pOnlineRoutingIn) irouting.ProvideManyRouter {
routers := in.Routers
sort.SliceStable(routers, func(i, j int) bool {
return routers[i].Priority < routers[j].Priority
```

```
})

var cRouters []*routinghelpers.ParallelRouter

for _, v := range routers {

cRouters = append(cRouters, &routinghelpers.ParallelRouter{

IgnoreError: true,

DoNotWaitForSearchValue: true,

Router: v.Routing,

})

}

return routinghelpers.NewComposableParallel(cRouters)

}
```

라우터 우선순위

| 라우터 타입 | 우선순위 | 특징 |

|----------|---------|------|

| Pubsub Router | 100 | IPNS 레코드 실시간 전파 |

| DHT Router | 1000 | 기본 분산 라우팅 |

| HTTP Router | 다양 | 빠른 중앙화된 검색 |

| Offline Router | 10000 | 오프라인 모드용 |

자동 릴레이 피더

IPFS는 NAT 뒤의 노드들을 위한 자동 릴레이 시스템을 제공합니다:

```
func autoRelayFeeder(cfgPeering config.Peering, peerChan chan<- peer.AddrInfo)
```

```
// 신뢰할 수 있는 피어들을 릴레이 후보로 제공
for _, trustedPeer := range cfgPeering.Peers {
if len(trustedPeer.Addrs) == 0 {
continue
}
select {
case peerChan <- trustedPeer:
case <-ctx.Done():
return
}
}
// DHT를 통해 발견된 가까운 피어들도 릴레이 후보로 추가
if dht != nil {
closestPeers, err := dht.WAN.GetClosestPeers(ctx, h.ID().String())
// 릴레이 후보로 추가...
}
}
```

성능 최적화 전략

1. 연결 관리 최적화

- **연결 수 제한**: 과도한 연결로 인한 리소스 낭비 방지

- **연결 품질 평가**: 지연시간과 대역폭을 고려한 피어 선택
- **주기적 정리**: 비활성 연결의 정리로 리소스 효율성 향상

2. 캐싱 전략

- **피어 정보 캐싱**: 자주 사용되는 피어 정보의 로컬 캐싱
- **라우팅 테이블 최적화**: 효율적인 라우팅 테이블 구조 유지
- **TTL 관리**: 적절한 생존 시간 설정으로 정보 신선도 유지

3. 네트워크 적응

- **대역폭 감지**: 네트워크 상황에 따른 동적 조정
- **지연시간 최적화**: 지리적으로 가까운 피어 우선 선택
- **부하 분산**: 인기 있는 콘텐츠의 부하 분산

문제 해결 가이드

일반적인 발견 문제

1. **Bootstrap 연결 실패**

- 네트워크 연결 상태 확인
- 방화벽 설정 검토
- 대안 bootstrap 피어 설정

2. **DHT 참여 실패**

- NAT 설정 확인
- 포트 포워딩 설정
- 릴레이 서비스 활용

3. **로컬 발견 실패**

- mDNS 지원 여부 확인
- 네트워크 권한 설정
- 방화벽 멀티캐스트 허용

디버깅 도구

```
# 연결된 피어 확인
ipfs swarm peers

# Bootstrap 피어 목록 확인
ipfs bootstrap list

# DHT 라우팅 테이블 확인
ipfs dht query <peer-id>

# 피어 발견 로그 활성화
export IPFS_LOGGING=debug
```

보안 고려사항

1. 신뢰성 검증

- **피어 신원 확인**: 공개키 기반 신원 검증
- **주소 검증**: 연결 가능한 주소인지 확인

- **평판 시스템**: 피어의 과거 행동 기반 신뢰도 평가

2. 공격 방어

- **Sybil 공격 방어**: 동일한 엔티티의 다중 신원 생성 방지
- **Eclipse 공격 방어**: 악의적 피어들에 의한 네트워크 격리 방지
- **DDoS 방어**: 과도한 연결 요청으로부터 보호

3. 프라이버시 보호

- **메타데이터 최소화**: 불필요한 정보 노출 방지
- **익명성 보장**: 가능한 경우 익명 연결 지원
- **트래픽 분석 방어**: 패턴 분석을 통한 정보 유출 방지

이 문서는 IPFS의 피어 발견 메커니즘에 대한 포괄적인 이해를 제공합니다. 다음 문서에서는 Bitswap 프로토콜의 동작 원리에 대해 자세히 다룰 예정입니다.